Weekly Review
R114 - OOP 1

Complete the weekly review, you'll have 15 minutes.

Weekly Review
R114 - OOP 1

Mark that review!

Once you've done it:

1. Submit the score into the Google Form.
2. Update your weekly review tracker with **targets**.

Starter

For the following statements, determine whether they are true or false:

1. The Private access modifier means that the variable/method can only be accessed anywhere within the current C# project.
2. The call stack contains a stack frame for each procedure and function in the code.
3. An implicit or default constructor is created automatically when the programmer does not make one themselves.

Starter

For the following statements, determine whether they are true or false:

1. The Private access modifier means that the variable/method can only be accessed anywhere within the current C# project. False
2. The call stack contains a stack frame for each procedure and function in the code. False
3. An implicit or default constructor is created automatically when the programmer does not make one themselves. True

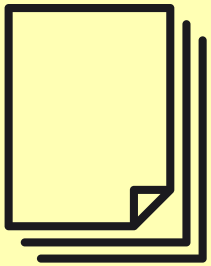
Inheritance

Inher

4.1.2.3 Object-oriented programming

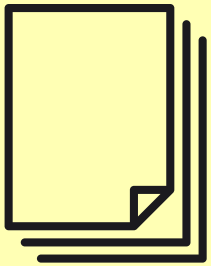
Content	Additional information
Be familiar with the concept of inheritance.	
Know why the object-oriented paradigm is used.	
Be able to write object-oriented programs.	Practical experience of coding for user-defined classes involving: • inheritance • public, private **and protected** specifiers.
Be able to draw and interpret class diagrams.	Class diagrams involving **single inheritance**, composition (black diamor line), aggregation (white diamond line), public (+), private (-) **and protected (#)** specifiers.

Inheritance



Inheritance is the act of having a class **inherit** all of the proprties and behaviours of another class.

Inheritance

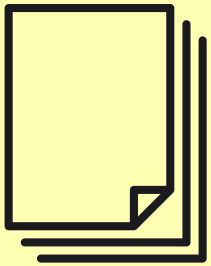


Inheritance is the act of having a class **inherit** all of the proprties and behaviours of another class.

It lets us re-implement code, without re-writing it!

Why might that be useful?

Inheritance



Inheritance is the act of having a class **inherit** all of the proprties and behaviours of another class.

It lets us re-implement code, without re-writing it!

Why might that be useful?

1. Time
2. Easier to read
3. Teamwork
4. Futureproofing

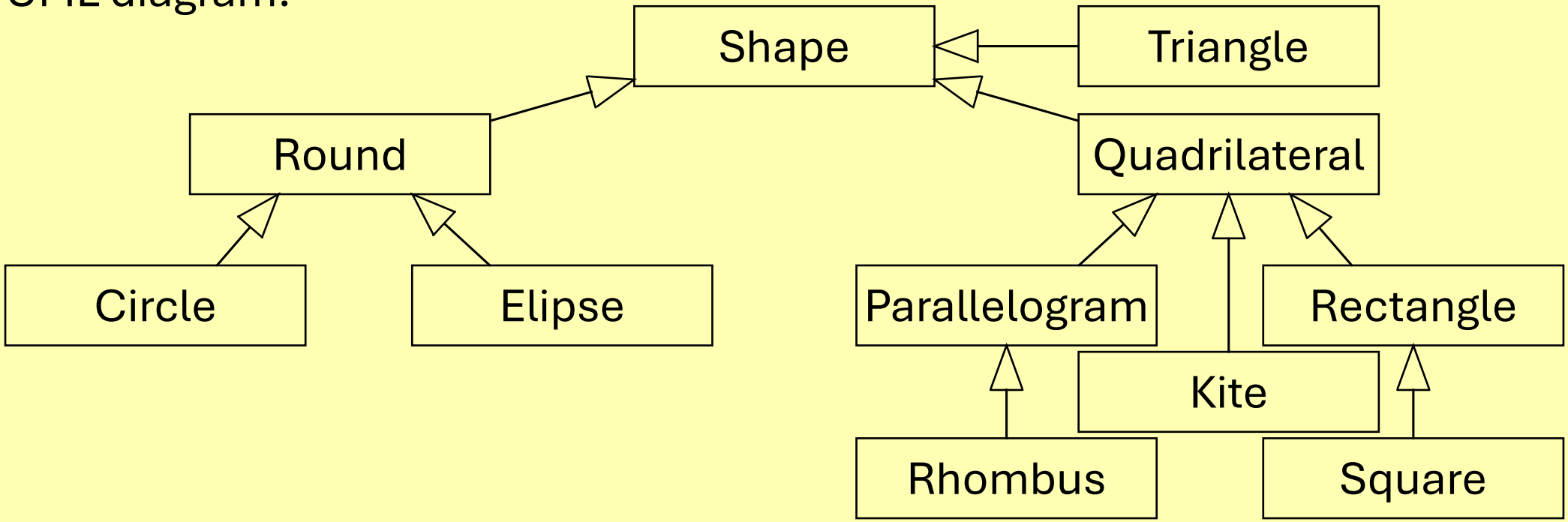
Mathematical Inheritance

Let's look at shapes, which we **should** all be comfortable with. How would you sort them? I want to think about 'X is a type of Y' relationships.

Square	Circle	Rectangle	Triangle
Parallelogram	Rhombus	Kite	Trapezium
Quadrilateral	Elipse		

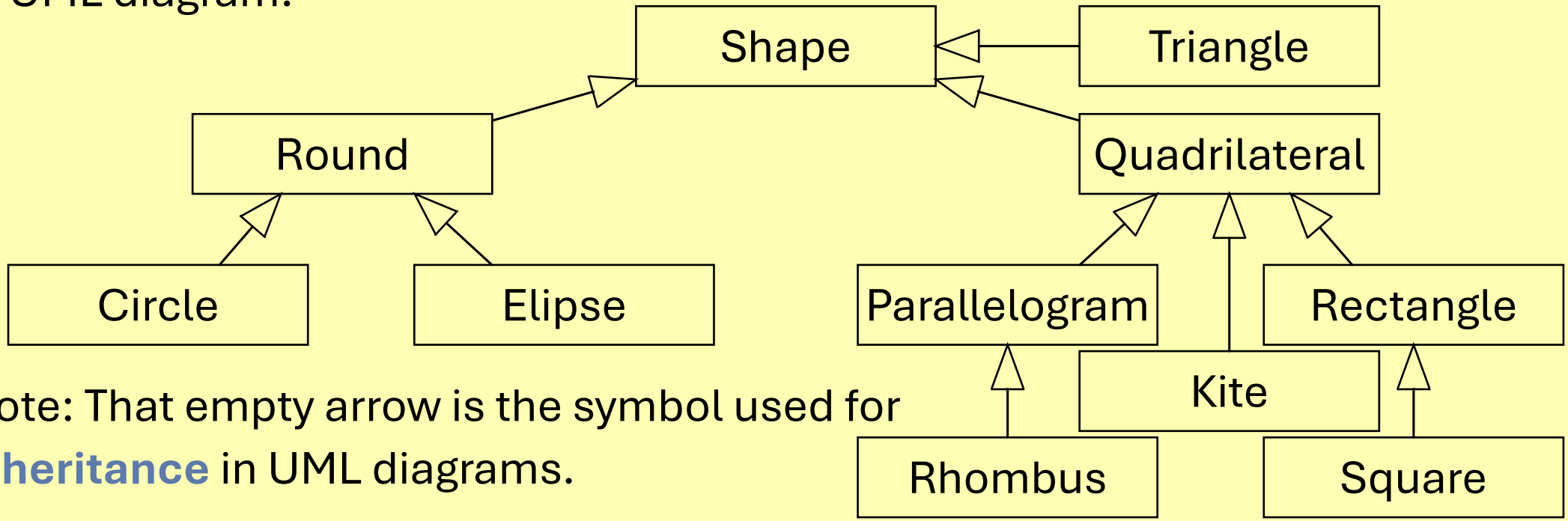
UML Inheritance

With the shapes from earlier, we could construct the following UML diagram:



UML Inheritance

With the shapes from earlier, we could construct the following UML diagram:



Note: That empty arrow is the symbol used for **inheritance** in UML diagrams.

Example Time!

Let's say we want to make three different things turn up on our screen:

1. A Player.
2. Enemies.
3. Collectibles

We could code each of these with a `Draw()`, as well as positions.

Example Time!

Let's say we want to make three different things turn up on our screen:

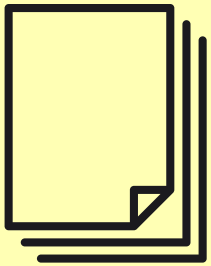
1. A Player.
2. Enemies.
3. Collectibles

We could code each of these with a `Draw()`, as well as positions.

If we wanted to change the program later, to store 3D coordinates, we'd need to update every one of these classes.

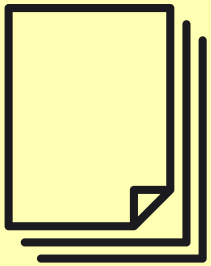
We'd also have to do massive amounts of re-coding if we wanted to move to a different output system (EG. MonoGame, WinForms, Maui)

The Solution



The solution is to have a **parent class** called `Entity` that will store the basic location and `Draw()` functions, and then write dedicated code for the remaining features of the other *types* of `Entity`.

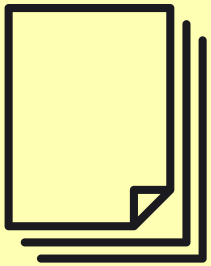
The Solution



The solution is to have a **parent class** called `Entity` that will store the basic location and `Draw()` functions, and then write dedicated code for the remaining features of the other *types* of `Entity`.

1. Let's draw the UML diagram.

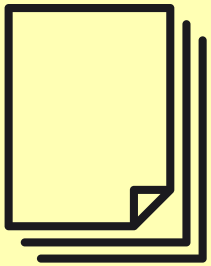
The Solution



The solution is to have a **parent class** called `Entity` that will store the basic location and `Draw()` functions, and then write dedicated code for the remaining features of the other *types* of `Entity`.

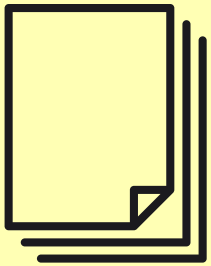
1. Let's draw the UML diagram.
2. Let's code it!

To Recap



1. **Inheritance**
   - Allows for sharing information/methods between classes.
   - Each class can only inherit from **one** other class
     (aka the parent / superclass)
   - a class can be inheritted by any number of classes
     (aka children / subclasses)

To Recap



1. **Inheritance**
   - Allows for sharing information/methods between classes.
   - Each class can only inherit from **one** other class (aka the parent / superclass)
   - a class can be inheritted by any number of classes (aka children / subclasses)
2. **Access Modifiers**
   - **public** (+) Accessible anywhere within the program.
   - **protected** (#) Accessible in the class and **derived classes**.
   - **private** (-) Accessible **only** within the defining class.

Practice Time!

W130 - C# Inheritance from BPCompSci

Extension: Create your own custom Exception called `AccountWithdrawalException`. It should inherit from the `Exception` base class. Pass in a reference to the account owner in the constructor, and add a `getOwnerName()` function. Test that exception.